

KCPC-CS250

Problem Sheet

Segment Trees: Queries, Updates, and Lazy Propagation

Developed by the KCPC Internal Committee, this problem sheet follows the *Segment Trees* workshop. Problems are divided into four categories:

- Warm-up practice set
 - Core practice set (to be attempted during the workshop)
 - Extended homework
 - Further challenge problems
-

I. Warm-up Practice Set

These problems are intended to build familiarity with the basic segment tree workflow: choosing what each node stores, writing the merge operation, and handling simple queries and updates.

1. **LeetCode 307 - Range Sum Query - Mutable**

Canonical first segment tree problem. Supports point updates and range sum queries. If you already know Fenwick trees, this is a good chance to compare the two data structures.

<https://leetcode.com/problems/range-sum-query-mutable/>

2. **AtCoder Library Practice Contest J - Segment Tree**

Very clean implementation exercise. In addition to point updates and range maximum queries, this problem also introduces tree descent / binary search on the segment tree.

https://atcoder.jp/contests/practice2/tasks/practice2_j

3. **CSES 1647 - Static Range Minimum Queries**

Strictly speaking, this is a static query problem, so a sparse table is often the intended solution. It is included here as a useful contrast case: when are segment trees appropriate, and when are they overkill?

<https://cses.fi/problemset/task/1647/>

II. Core Workshop Practice Set

These problems should be attempted during the workshop. They reinforce the standard segment tree patterns that appear repeatedly in contests: point updates, different merge operations, and richer node states.

For any questions that were not completed during the workshop, please attempt them independently afterwards.

[REDACTED]’s **Note**. Segment trees are less about memorising one template and more about identifying the correct *node information*, *merge operation*, and, when needed, *lazy tag composition*. We will emphasise this design process in the live walk-throughs.

1. **CSES 1648 - Dynamic Range Sum Queries**

Textbook segment tree problem. Build the tree, support point updates, and answer range sum queries correctly and efficiently.

<https://cses.fi/problemset/task/1648/>

2. **CSES 1650 - Range Xor Queries**

Same general structure as range sum queries, but with xor as the merge operation. A good reminder that the segment tree is really an abstraction over an associative operation.

<https://cses.fi/problemset/task/1650/>

3. **CSES 1190 - Subarray Sum Queries**

Classic segment tree problem. Each node stores four values: total sum, best prefix, best suffix, and best subarray sum. This is one of the most important examples of storing more than one value per node.

<https://cses.fi/problemset/task/1190/>

III. Extended Homework

Important. The following problems are intended to be more extensive or conceptually harder. They should only be attempted after completing the warm-up and core sets. They are designed to introduce lazy propagation, non-trivial update rules, and more advanced pruning ideas.

1. **CSES 1651 - Range Update Queries**

A good first step beyond point updates. While it can be approached with lazy propagation, there are also simpler ways to think about it if you examine what the query type is really asking for.

<https://cses.fi/problemset/task/1651/>

2. **CSES 1735 - Range Updates and Sums**

Standard lazy propagation problem with multiple update types. You must handle range addition, range assignment, and range sum queries. Tag composition is the main difficulty.

<https://cses.fi/problemset/task/1735/>

3. **Codeforces 438D - The Child and Sequence**

Very important pruning idea. Modulo updates only matter on segments whose maximum is at least the modulus, so storing segment maxima gives a powerful optimisation. This is a gentle introduction to “segment tree beats” style reasoning.

<https://codeforces.com/problemset/problem/438/D>

4. **AtCoder Library Practice Contest K - Range Affine Range Sum**

Strong lazy propagation exercise. Each update applies an affine transform $x \mapsto ax + b$, so both the stored node values and the lazy tags must be composed carefully.

https://atcoder.jp/contests/practice2/tasks/practice2_k

IV. Further Challenge Problems

These are classical segment tree problems ranging from comfortable intermediate practice to very advanced implementations. Some overlap with the homework section above; they are retained here because they are standard references for the topic.

1. **SPOJ - KQUERY** *[Persistent segment tree / Merge sort tree]*
<http://www.spoj.com/problems/KQUERY/>
2. **Codeforces 339D - Xenia and Bit Operations**
<https://codeforces.com/problemset/problem/339/D>
3. **UVA 11402 - Ahoy, Pirates!**
https://uva.onlinejudge.org/index.php?option=com_onlinejudge&Itemid=8&page=show_problem&problem=2397
4. **SPOJ - GSS3**
<http://www.spoj.com/problems/GSS3/>
5. **Codeforces 1234D - Distinct Characters Queries**
<https://codeforces.com/problemset/problem/1234/D>
6. **Codeforces 356A - Knight Tournament** *[For beginners]*
<https://codeforces.com/contest/356/problem/A>
7. **Codeforces 474F - Ant colony**
<https://codeforces.com/contest/474/problem/F>
8. **Codeforces 515E - Drazil and Park**
<https://codeforces.com/contest/515/problem/E>
9. **Codeforces 52C - Circular RMQ**
<https://codeforces.com/problemset/problem/52/C>
10. **Codeforces 121E - Lucky Array**
<https://codeforces.com/contest/121/problem/E>
11. **Codeforces 438D - The Child and Sequence** *[Also listed in Homework]*
<https://codeforces.com/contest/438/problem/D>
12. **Codeforces 446C - DZY Loves Fibonacci Numbers** *[Lazy propagation]*
<https://codeforces.com/contest/446/problem/C>
13. **Codeforces 610E - Alphabet Permutations**
<https://codeforces.com/problemset/problem/610/E>
14. **Codeforces 895E - Eyes Closed**
<https://codeforces.com/problemset/problem/895/E>
15. **Codeforces 580E - Kefa and Watch**
<https://codeforces.com/problemset/problem/580/E>
16. **Codeforces 558E - A Simple Task**

<https://codeforces.com/problemset/problem/558/E>

17. **Codeforces 920F - SUM and REPLACE**

<https://codeforces.com/problemset/problem/920/F>

18. **Codeforces 242E - XOR on Segment**

[Lazy propagation]

<https://codeforces.com/problemset/problem/242/E>

19. **Codeforces 1114F - Please, another Queries on Array?**

[Lazy propagation]

<https://codeforces.com/problemset/problem/1114/F>

20. **COCI - Deda**

[Last element smaller or equal to x / Binary search]

https://oj.uz/problem/view/COCI17_deda

21. **Codeforces 869E - The Untended Antiquity**

[2D]

<https://codeforces.com/problemset/problem/869/E>

22. **CSES 1143 - Hotel Queries**

<https://cses.fi/problemset/task/1143>

23. **CSES 1736 - Polynomial Queries**

<https://cses.fi/problemset/task/1736>

24. **CSES 1735 - Range Updates and Sums**

[Also listed in Homework]

<https://cses.fi/problemset/task/1735>

V. Reading List

The following references are strongly recommended.

- **Codeforces blog: Segment tree for graph problems**
<https://codeforces.com/blog/entry/135761>
 - **AtCoder Library documentation: segtree**
Excellent reference for thinking about segment trees in terms of monoids.
https://atcoder.github.io/ac-library/master/document_en/segtree.html
 - **Codeforces EDU / ITMO Academy: Segment Tree lesson**
Structured tutorial sequence with many progressively harder tasks.
<https://codeforces.com/edu/course/2/lesson/4>
 - **cp-algorithms: Segment Tree**
Reliable reference for standard implementations and common variants.
https://cp-algorithms.com/data_structures/segment_tree.html
-

Guidance:

- Before coding, define the *node state*, the *merge operation*, and, if applicable, the *lazy tag* and how tags compose.
- Check whether the operation you are using is associative. Segment trees rely on this heavily.
- Be consistent with interval convention: either use $[l, r]$ everywhere or $[l, r)$ everywhere.

- For lazy propagation, composition order matters. “Apply A then B” is generally different from “apply B then A”.
- Many tree-descent queries require a monotone property. If your predicate is not monotone, a binary search on the segment tree may fail.
- Use `long long` when working with sums unless the constraints make overflow impossible.
- Do not default to a segment tree automatically. Sometimes a Fenwick tree, sparse table, prefix sums, or offline sorting is simpler and faster.